

# Web Application Security

**Dr. Raj K Jaiswal**

# Web and HTTP

*First, a quick review...*

- web page consists of *objects*, each of which can be stored on different Web servers
- object can be HTML file, JPEG image, Java applet, audio file,...
- web page consists of *base HTML-file* which includes *several referenced objects, each* addressable by a *URL*, e.g.,

`www.someschool.edu/someDept/pic.gif`

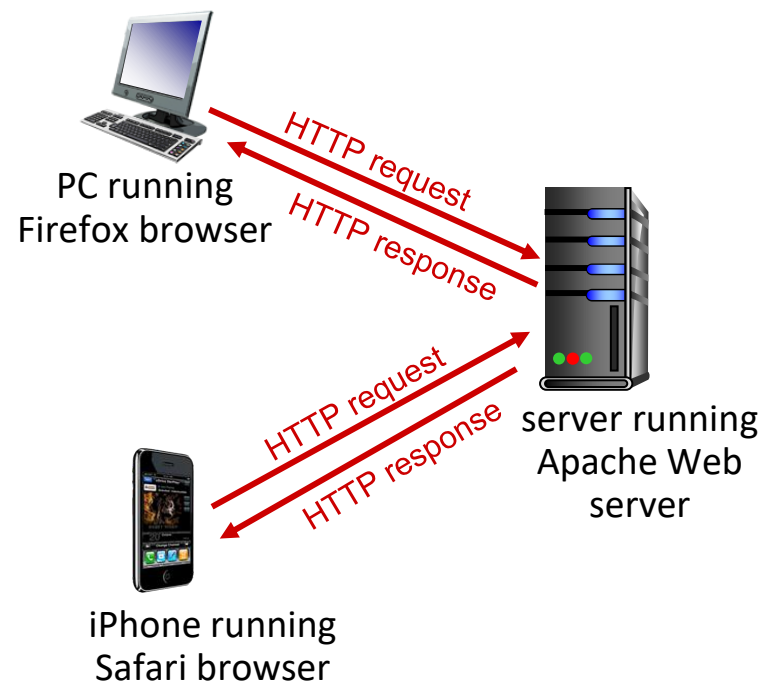
host name

path name

# HTTP overview

## HTTP: hypertext transfer protocol

- Web's application-layer protocol
- client/server model:
  - *client*: browser that requests, receives, (using HTTP protocol) and “displays” Web objects
  - *server*: Web server sends (using HTTP protocol) objects in response to requests



# HTTP overview (continued)

## *HTTP uses TCP:*

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

## *HTTP is “stateless”*

- server maintains *no* information about past client requests

*aside*  
protocols that maintain  
“state” are complex!

- past history (state) must be maintained
- if server/client crashes, their views of “state” may be inconsistent, must be reconciled

# HTTP connections: two types

## *Non-persistent HTTP*

1. TCP connection opened
2. at most one object sent over TCP connection
3. TCP connection closed

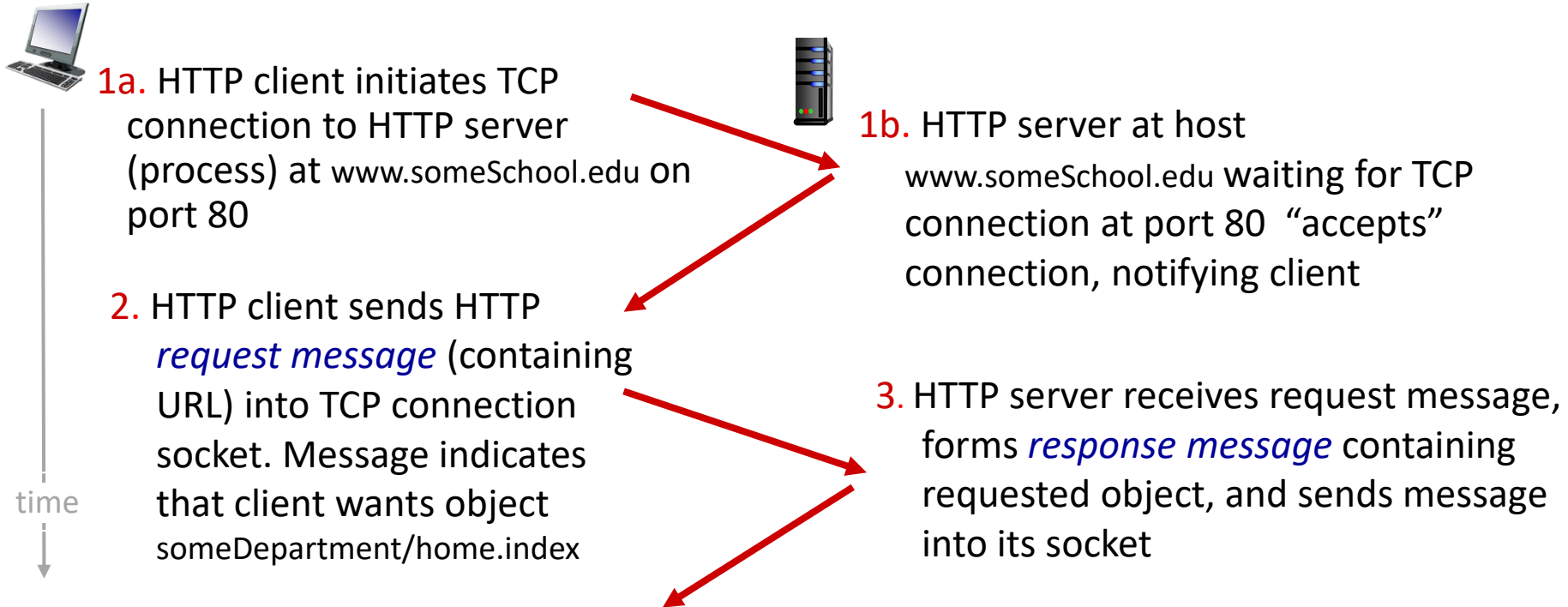
downloading multiple objects required multiple connections

## *Persistent HTTP*

- TCP connection opened to a server
- multiple objects can be sent over *single* TCP connection between client, and that server
- TCP connection closed

# Non-persistent HTTP: example

User enters URL: `www.someSchool.edu/someDepartment/home.index`  
(containing text, references to 10 jpeg images)



# Non-persistent HTTP: example (cont.)

User enters URL: `www.someSchool.edu/someDepartment/home.index`  
(containing text, references to 10 jpeg images)



5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

4. HTTP server closes TCP connection.

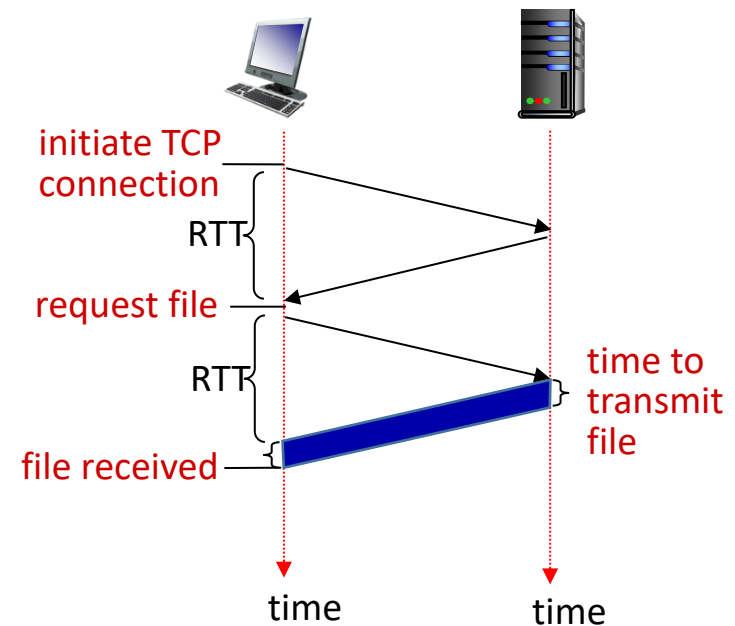


# Non-persistent HTTP: response time

**RTT (definition):** time for a small packet to travel from client to server and back

**HTTP response time (per object):**

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object/file transmission time



*Non-persistent HTTP response time = 2RTT + file transmission time*

# Persistent HTTP (HTTP 1.1)

## *Non-persistent HTTP issues:*

- requires 2 RTTs per object
- OS overhead for *each* TCP connection
- browsers often open multiple parallel TCP connections to fetch referenced objects in parallel

## *Persistent HTTP (HTTP1.1):*

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects (cutting response time in half)

# HTTP request Message

- two types of HTTP messages: *request, response*
- HTTP request message:
  - ASCII (human-readable format)

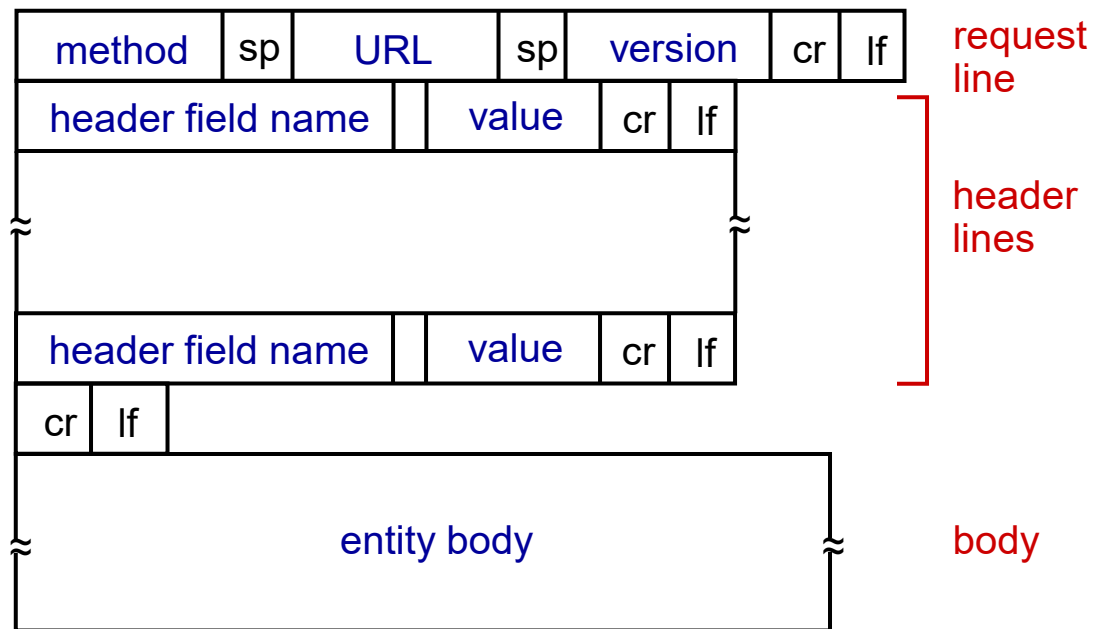
request line (GET, ,  
POST,  
HEAD commands)

 carriage return character  
line-feed character

carriage return, line  
feed at start of line  
indicates end of header  
lines 

\* Check out the online interactive exercises for more  
examples: [http://gaia.cs.umass.edu/kurose\\_ross/interactive/](http://gaia.cs.umass.edu/kurose_ross/interactive/)

# HTTP request message: general format



# Other HTTP request Messages

## POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

## GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):  
`www.somesite.com/animalsearch?monkeys&banana`

## HEAD method:

- requests headers (only) that would be returned *if* specified URL were requested with an HTTP GET method.

## PUT method:

- uploads new file (object) to server
- completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

# HTTP response Message

status line (protocol  HTTP/1.1 200 OK  
status code status phrase)

# HTTP response Status Codes

- status code appears in 1st line in server-to-client response message.
- some sample codes:

## 200 OK

- request succeeded, requested object later in this message

## 301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

## 400 Bad Request

- request msg not understood by server

## 404 Not Found

- requested document not found on this server

## 505 HTTP Version Not Supported

# Trying out HTTP (client side) for yourself

## 1. netcat to your favorite Web server:

% nc -c -v gaia.cs.umass.edu 80 (for Mac)

>ncat -C gaia.cs.umass.edu 80 (for Windows)

- opens TCP connection to port 80 (default HTTP server port) at gaia.cs.umass.edu.
- anything typed in will be sent to port 80 at gaia.cs.umass.edu

## 2. type in a GET HTTP request:

**GET /kurose\_ross/interactive/index.php HTTP/1.1**

**Host: gaia.cs.umass.edu**

- by typing this in (hit carriage return twice), you send this minimal (but complete) GET request to HTTP server

## 3. look at response message sent by HTTP server!

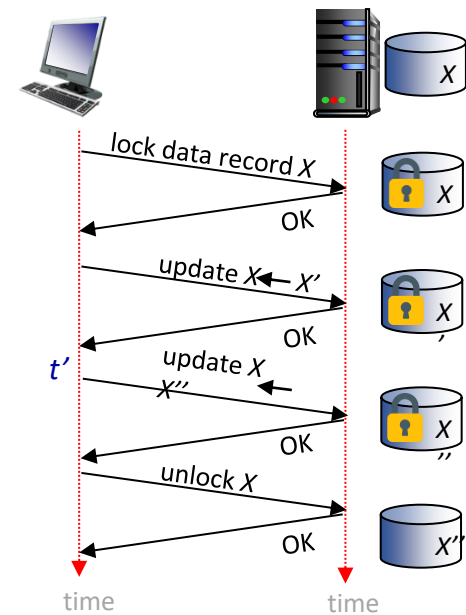
(or use Wireshark to look at captured HTTP request/response)

# Maintaining user/server state: cookies

Recall: HTTP GET/response interaction is *stateless*

- no notion of multi-step exchanges of HTTP messages to complete a Web “transaction”
  - no need for client/server to track “state” of multi-step exchange
  - all HTTP requests are independent of each other
  - no need for client/server to “recover” from a partially-completed-but-never-completely-completed transaction

a **stateful protocol**: client makes two changes to  $X$ , or none at all



**Q:** what happens if network connection or client crashes at  $t'$  ?

# Maintaining user/server state: cookies

Web sites and client browser use *cookies* to maintain some state between transactions

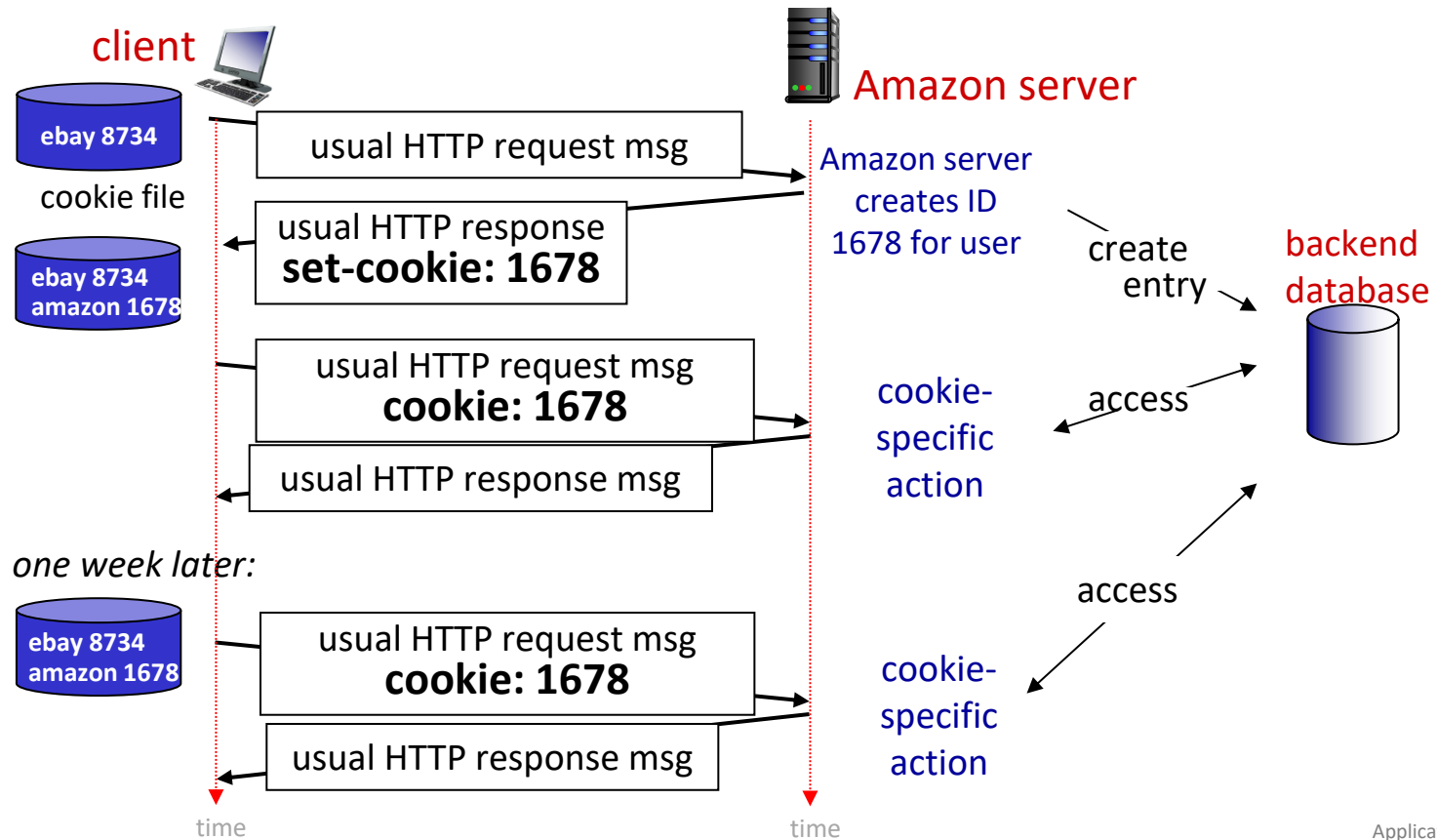
## *four components:*

- 1) cookie header line of HTTP *response* message
- 2) cookie header line in next HTTP *request* message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at Web site

## Example:

- Susan uses browser on laptop, visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
  - unique ID (aka “cookie”)
  - entry in backend database for ID
- subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to “identify” Susan

# Maintaining user/server state: cookies



# HTTP cookies: comments

## *What cookies can be used for:*

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

## *Challenge: How to keep state?*

- *at protocol endpoints:* maintain state at sender/receiver over multiple transactions
- *in messages:* cookies in HTTP messages carry state

aside

*cookies and privacy:*

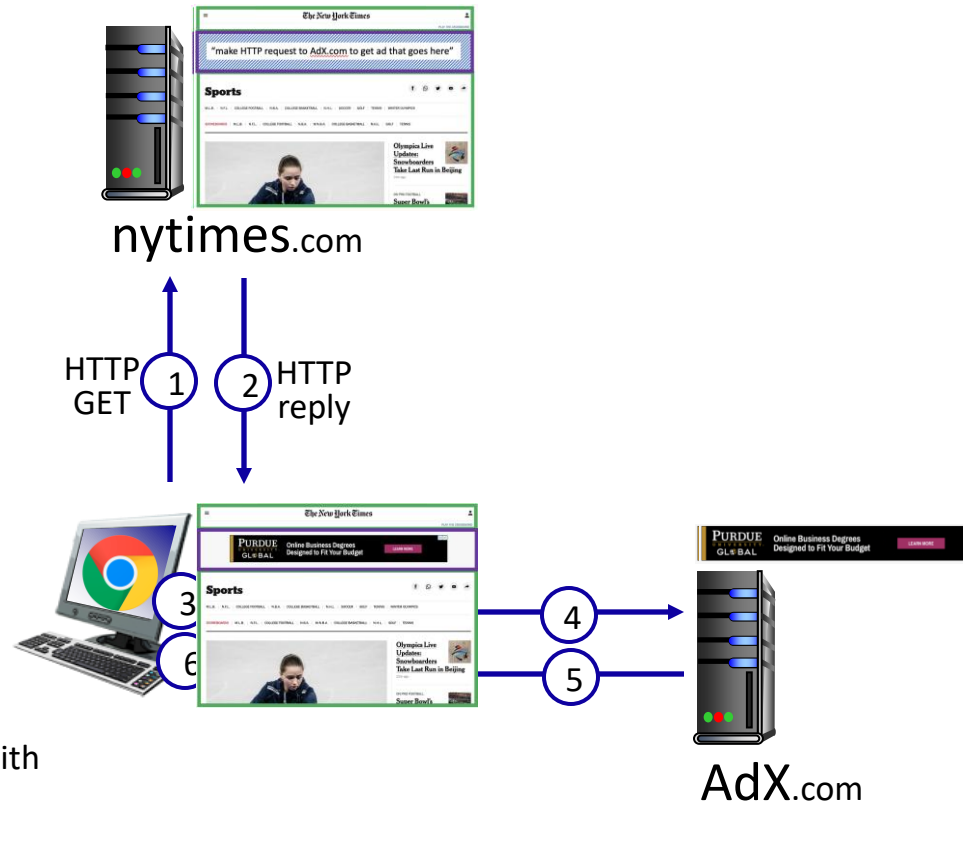
- cookies permit sites to *learn* a lot about you on their site.
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

# Example: displaying a NY Times web page

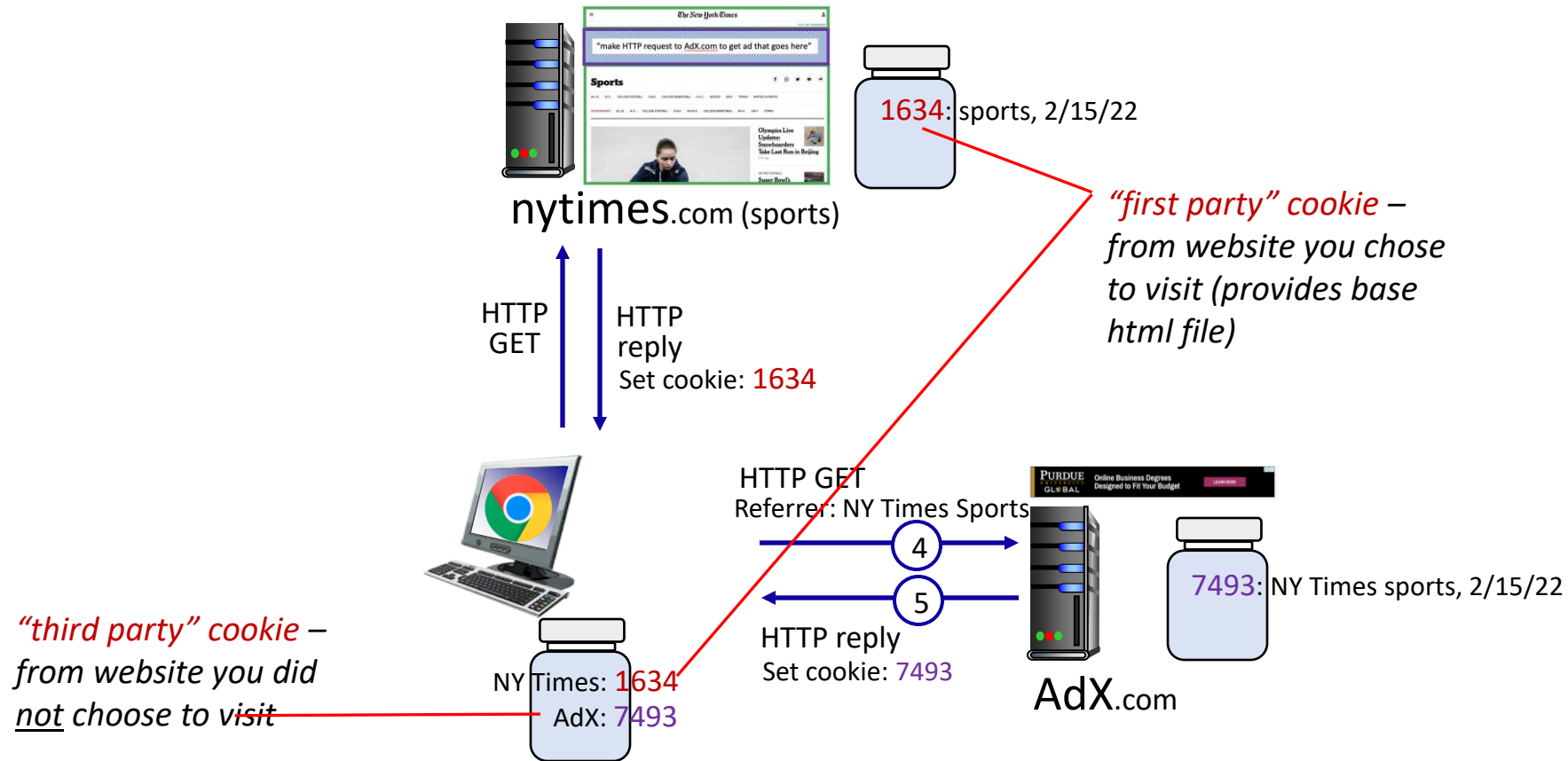
1 2 GET base html file  
from nytimes.com

4 5 fetch ad from  
AdX.com

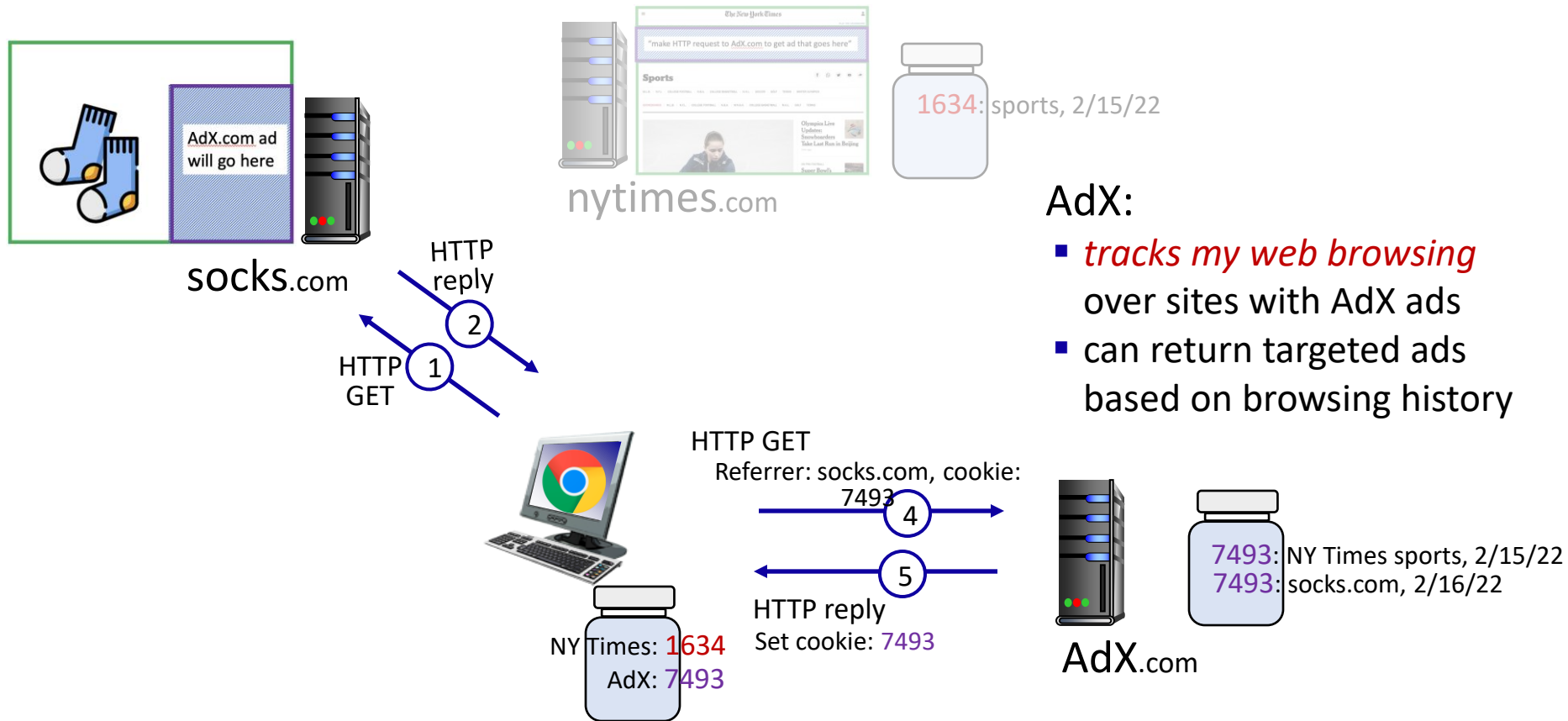
7 display composed  
page



# Cookies: tracking a user's browsing behavior



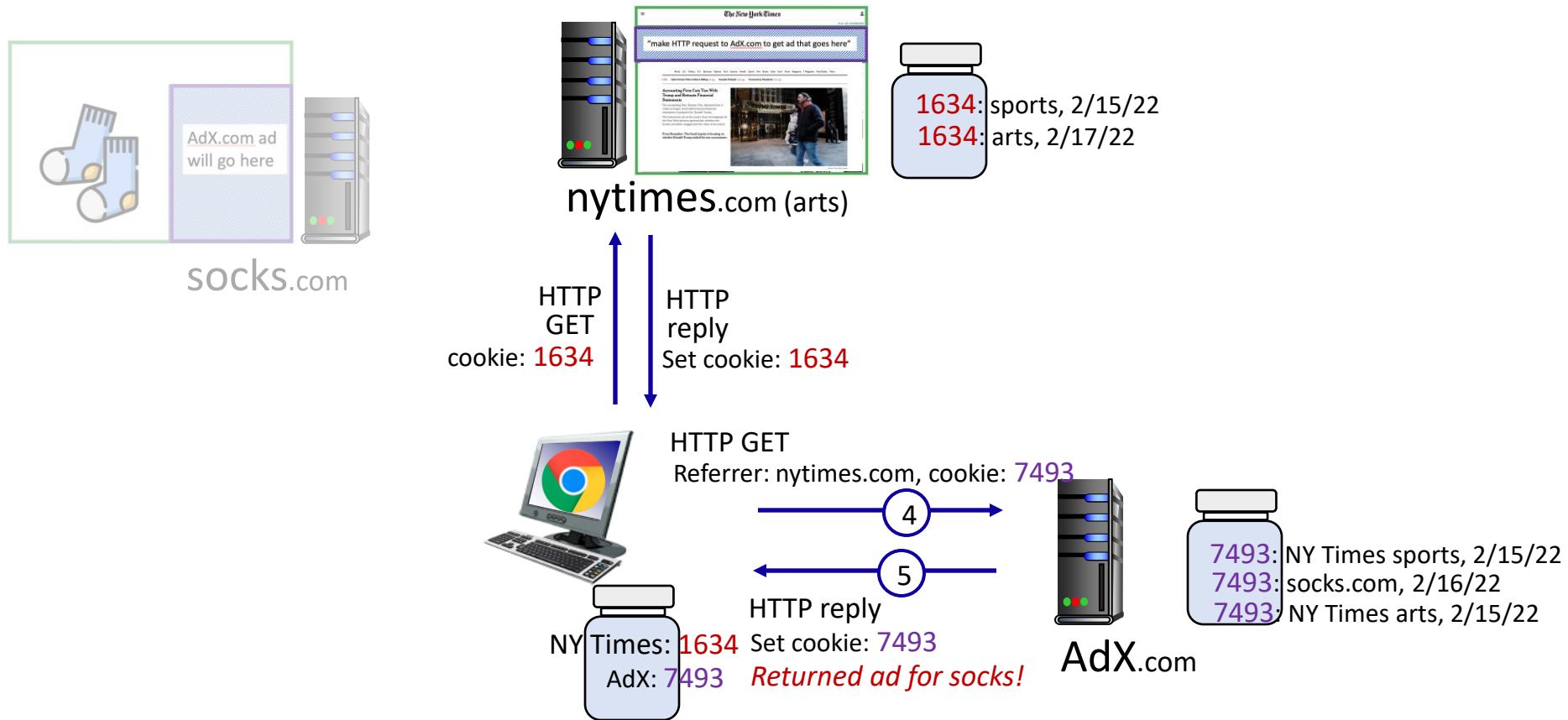
# Cookies: tracking a user's browsing behavior



AdX:

- *tracks my web browsing* over sites with AdX ads
- can return targeted ads based on browsing history

# Cookies: tracking a user's browsing behavior (one day later)



# Cookies: tracking a user's browsing behavior

Cookies can be used to:

- track user behavior on a given website (**first party cookies**)
- track user behavior across multiple websites (**third party cookies**) without user ever choosing to visit tracker site (!)
- tracking may be *invisible* to user:
  - rather than displayed ad triggering HTTP GET to tracker, could be an invisible link

third party tracking via cookies:

- disabled by default in Firefox, Safari browsers
- to be disabled in Chrome browser in 2023

# GDPR (EU General Data Protection Regulation) and cookies

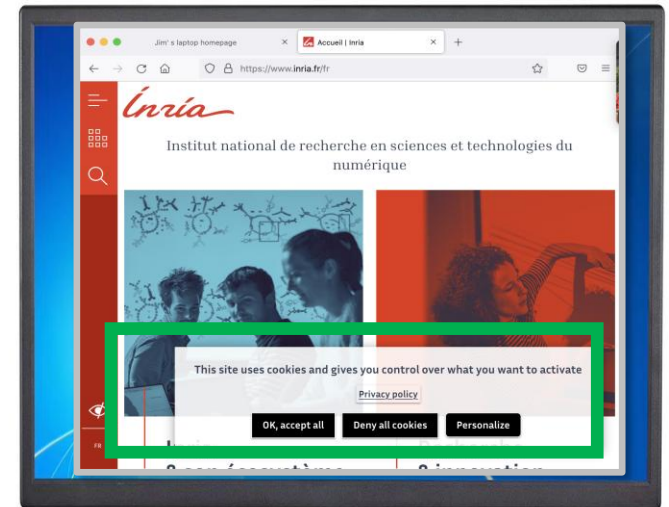
“Natural persons may be associated with online identifiers [...] such as internet protocol addresses, cookie identifiers or other identifiers [...].

This may leave traces which, in particular when combined with unique identifiers and other information received by the servers, may be used to create profiles of the natural persons and identify them.”

GDPR, recital 30 (May 2018)



when cookies can identify an individual, cookies are considered personal data, subject to GDPR personal data regulations

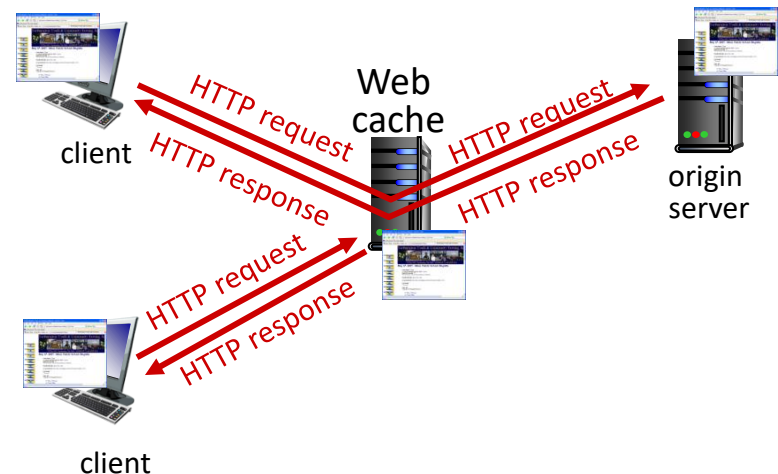


*User has explicit control over whether or not cookies are allowed*

# Web caches

*Goal:* satisfy client requests without involving origin server

- user configures browser to point to a (local) *Web cache*
- browser sends all HTTP requests to cache
  - *if* object in cache: cache returns object to client
  - *else* cache requests object from origin server, caches received object, then returns object to client



# Web caches (aka proxy servers)

- Web cache acts as both client and server
  - server for original requesting client
  - client to origin server
- server tells cache about object's allowable caching in response header:

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

## *Why* Web caching?

- reduce response time for client request
  - cache is closer to client
- reduce traffic on an institution's access link
- Internet is dense with caches
  - enables “poor” content providers to more effectively deliver content

# Caching example

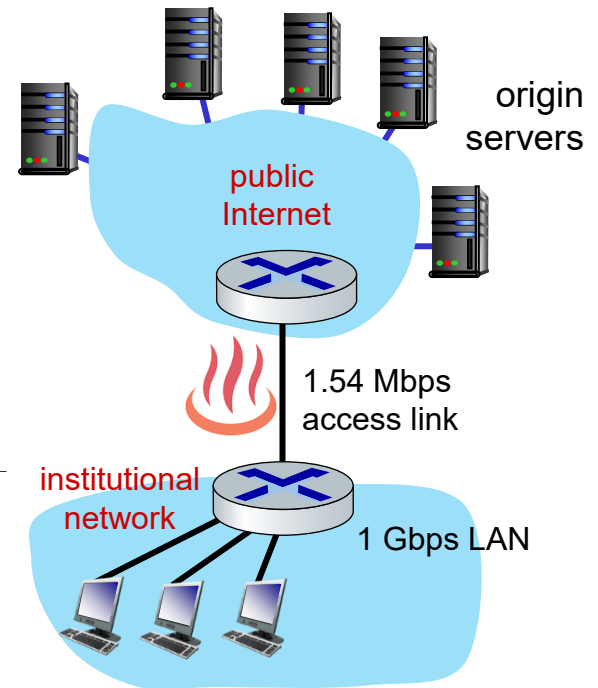
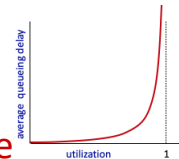
## Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
  - avg data rate to browsers: 1.50 Mbps

## Performance:

- access link utilization = .97
- LAN utilization: .0015
- end-end delay = Internet delay + access link delay + LAN delay  
= 2 sec + minutes + usecs

problem: large queueing delays at high utilization!



# Option 1: buy a faster access link

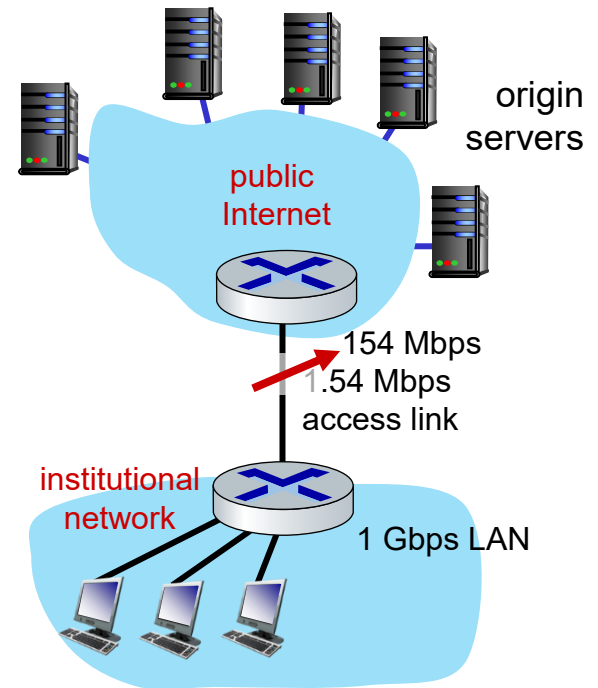
## Scenario:

- access link rate: ~~1.54~~ 154 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
  - avg data rate to browsers: 1.50 Mbps

## Performance:

- access link utilization = ~~.97~~ .0097
- LAN utilization: .0015
- end-end delay = Internet delay +  
access link delay + LAN delay  
= 2 sec + ~~minutes~~ + usecs

Cost: faster access link (expensive!) → msec



# Option 2: install a web cache

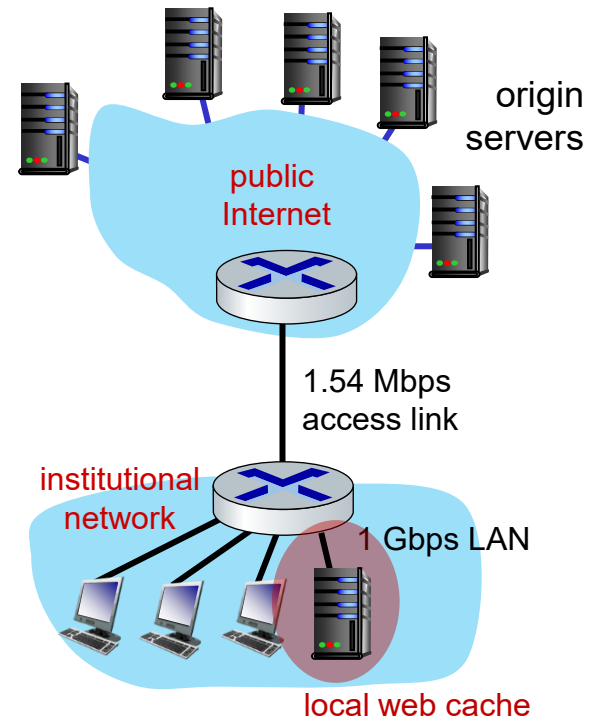
## Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
  - avg data rate to browsers: 1.50 Mbps

**Cost:** web cache (cheap!)

## Performance:

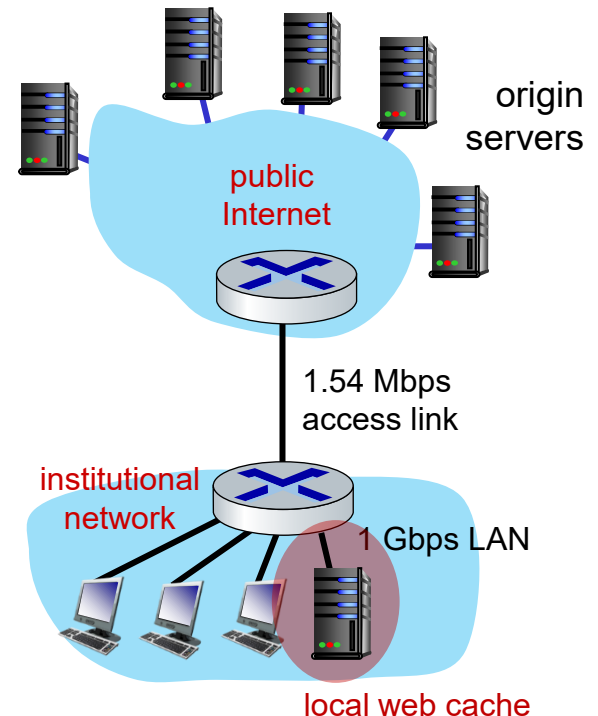
- LAN utilization: .?
  - access link utilization = ?
  - average end-end delay = ?
- How to compute link utilization, delay?*



# Calculating access link utilization, end-end delay with cache:

suppose cache hit rate is 0.4:

- 40% requests served by cache, with low (msec) delay
- 60% requests satisfied at origin
  - rate to browsers over access link  
 $= 0.6 * 1.50 \text{ Mbps} = .9 \text{ Mbps}$
  - access link utilization  $= 0.9 / 1.54 = .58$   
means low (msec) queueing delay at access link
- average end-end delay:  
 $= 0.6 * (\text{delay from origin servers})$   
 $+ 0.4 * (\text{delay when satisfied at cache})$   
 $= 0.6 (2.01) + 0.4 (\sim \text{msecs}) = \sim 1.2 \text{ secs}$

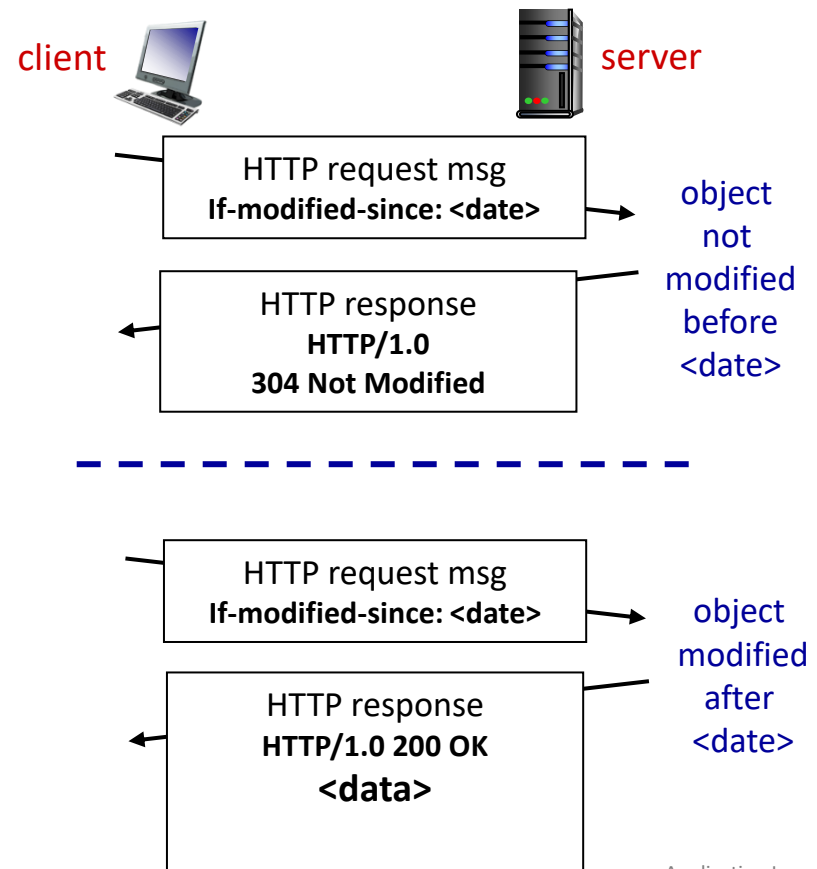


*lower average end-end delay than with 154 Mbps link (and cheaper too!)*

# Browser caching: Conditional GET

**Goal:** don't send object if browser has up-to-date cached version

- no object transmission delay (or use of network resources)
- **client:** specify date of browser-cached copy in HTTP request  
**If-modified-since: <date>**
- **server:** response contains no object if browser-cached copy is up-to-date:  
**HTTP/1.0 304 Not Modified**



# HTTP/2

*Key goal:* decreased delay in multi-object HTTP requests

HTTP1.1: introduced **multiple, pipelined GETs** over single TCP connection

- server responds *in-order* (FCFS: first-come-first-served scheduling) to GET requests
- with FCFS, small object may have to wait for transmission (**head-of-line (HOL) blocking**) behind large object(s)
- loss recovery (retransmitting lost TCP segments) stalls object transmission

# HTTP/2

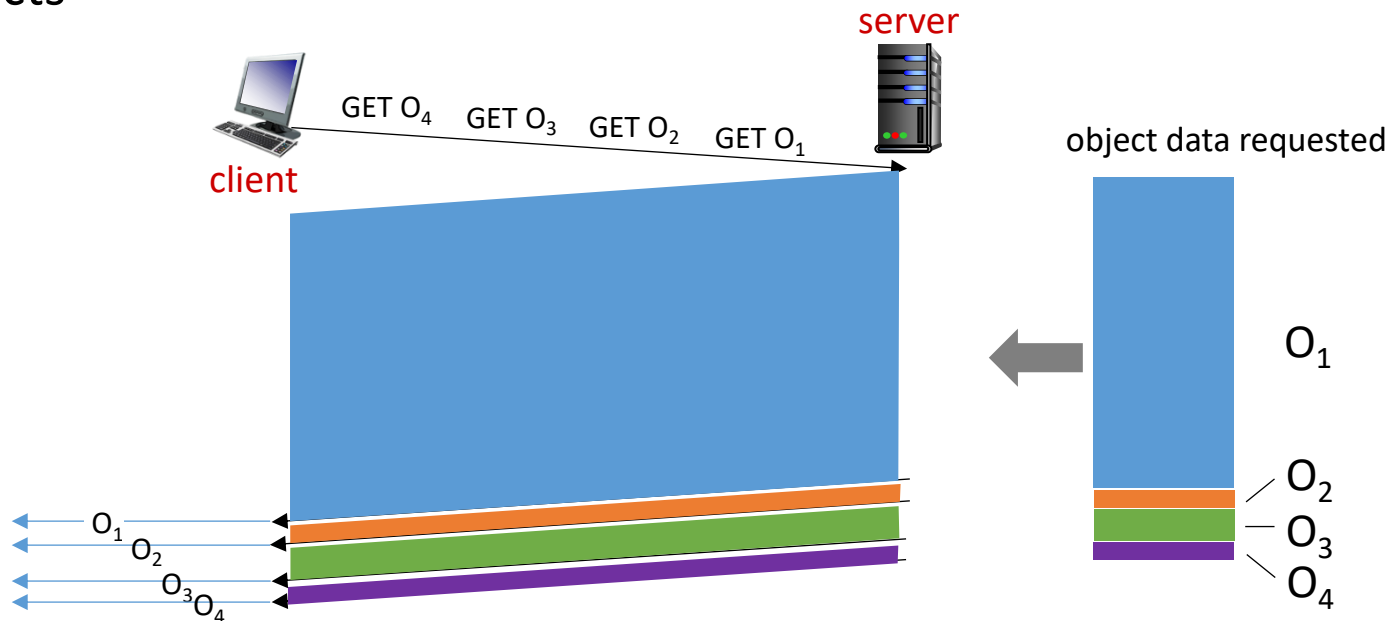
*Key goal:* decreased delay in multi-object HTTP requests

HTTP/2: [RFC 7540, 2015] increased flexibility at *server* in sending objects to client:

- methods, status codes, most header fields unchanged from HTTP 1.1
- transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
- *push* unrequested objects to client
- divide objects into frames, schedule frames to mitigate HOL blocking

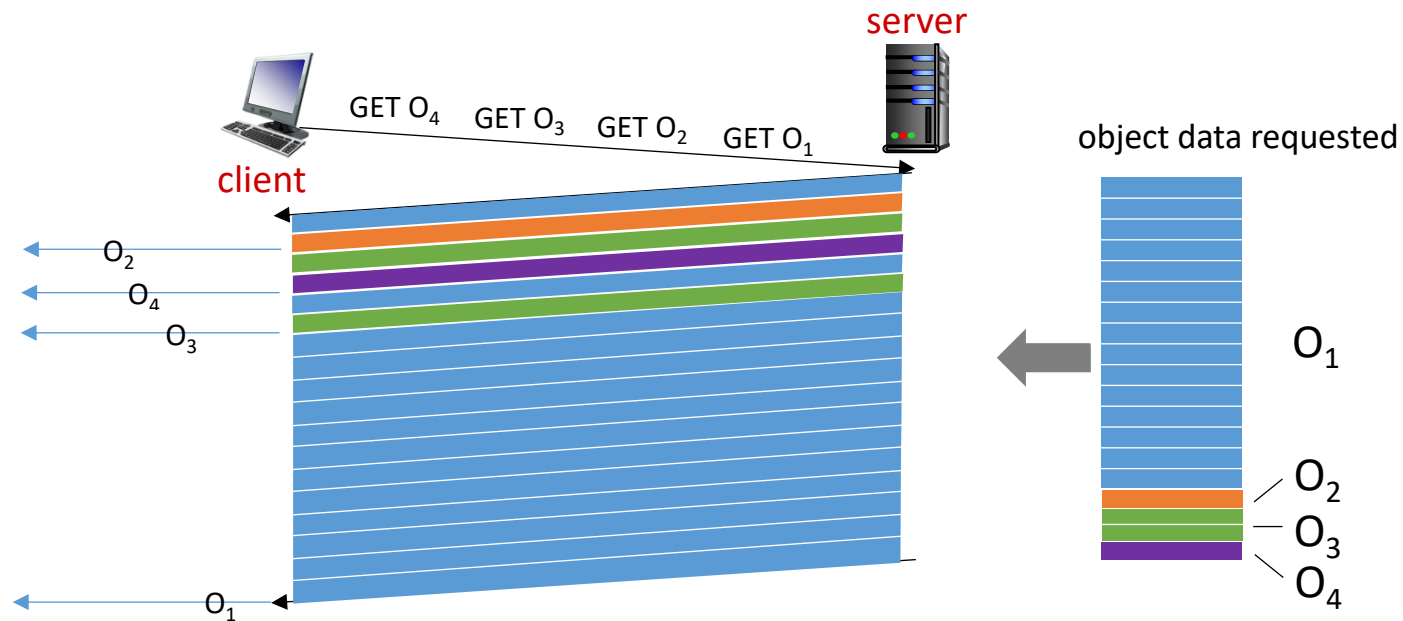
# HTTP/2: mitigating HOL blocking

HTTP 1.1: client requests 1 large object (e.g., video file) and 3 smaller objects



# HTTP/2: mitigating HOL blocking

HTTP/2: objects divided into frames, frame transmission interleaved



*$O_2, O_3, O_4$  delivered quickly,  $O_1$  slightly delayed*

# HTTP/2 to HTTP/3

HTTP/2 over single TCP connection means:

- recovery from packet loss still stalls all object transmissions
  - as in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
- no security over vanilla TCP connection
- **HTTP/3**: adds security, per object error- and congestion-control (more pipelining) over UDP
  - more on HTTP/3 in transport layer

# Same Origin Policy

- The basic security model enforced in the browser
- SoP isolates the scripts and resources downloaded from different origins
  - E.g., evil.org scripts cannot access bank.com resources
- Use origin as the security principal
  - Note that the concept of user accounts does not apply here as security principals
- Origin = domain name + protocol + port
  - all three must be equal for origin to be considered the same

# Same Original Policy: What it Controls

- Same-origin policy applies to the following accesses:
  - manipulating browser windows
  - URLs requested via the XMLHttpRequest
  - manipulating frames (including inline frames)
  - manipulating documents (included using the object tag)
  - manipulating cookies

# Problems with S-O Policy

- Poorly enforced on some browsers
  - Particularly older browsers
- Limitations if site hosts unrelated pages
  - Example: Web server often hosts sites for unrelated parties
    - <http://www.example.com/account/>
    - <http://www.example.com/otheraccount/>
  - Same-origin policy allows script on one page to access properties of document from another
- Can be bypassed in Cross-Site-Scripting attacks
- Usability: Sometimes prevents desirable cross-origin resource sharing

# Browser Architecture: One Process versus Multiple Processes

- Most processes (e.g., Firefox, Internet Explorer) use one process for a web browser
  - Multiple threads are used for rendering different webpages
- Chrome uses multiple processes
  - Use OS protection mechanism to ensure that webpages from different sites cannot easily interact
    - Because they run in different processes
  - Reliability advantage: crashing in rendering one website doesn't affect another
  - Security advantage: vulnerability in rendering does not compromise other sites; isolate plug-ins
  - Uses 3 types of processes: browser, renderers, plug-ins

# Encoding

- *The process of conversion of data from one form to another form is known as Encoding.*
- It is used to transform the data so that data can be supported and used by different systems.
- **Encoding in Electronics:** In electronics, encoding refers to converting analog signals to digital signals.
- **Encoding in Computing:** In computing, encoding is a process of converting data to an equivalent cipher by applying specific code, letters, and numbers to the data.

# Type of Encoding Technique

- **Character Encoding**
- **Image & Audio and Video Encoding**

## Character Encoding:

- It encodes characters into bytes.
- It informs the computers how to interpret the zero and ones into real characters, numbers, and symbols.
- The computer understands only binary data; hence it is required to convert these characters into numeric codes.
- To achieve this, each character is converted into binary code, and for this, text documents are saved with encoding types.
- It can be done by pairing numbers with characters.

# Type of Encoding Technique

- There are different types of Character Encoding techniques, which are given below:
  - **HTML Encoding**
  - **URL Encoding**
  - **Unicode Encoding**
  - **Base64 Encoding**
  - **Hex Encoding**
  - **ASCII Encoding**
- **HTML Encoding**
  - HTML encoding is used to display an HTML page in a proper format. With encoding, a web browser gets to know that which character set to be used.
  - In HTML, there are various characters used in HTML Markup such as <, >. To encode these characters as content, we need to use an encoding.

# Type of Encoding Technique

- URL Encoding
  - URL (Uniform resource locator) Encoding is used to convert characters in such a format that they can be transmitted over the internet.
  - It is also known as percent-encoding.
  - The URL Encoding is performed to send the URL to the internet using the ASCII character-set.
  - Non-ASCII characters are replaced with a %, followed by the hexadecimal digits.

# Type of Encoding Technique

- UNICODE Encoding

- Unicode is an encoding standard for a universal character set.
- Unicode is a list of characters with unique decimal numbers (code points). A = 65, B = 66, C = 67, ....
- This list of decimal numbers represent the string "hello": 104 101 108 108 111
- It allows encoding, represent, and handle the text represented in most of the languages or writing systems that are available worldwide.
- It provides a code point or number for each character in every supported language.
- It can represent approximately all the possible characters possible in all the languages.
- A particular sequence of bits is known as a coding unit.
- A UNICODE standard can use 8, 16, or 32 bits to represent the characters.
- The Unicode standard defines Unicode Transformation Format (UTF) to encode the code points.
- UNICODE Encoding standard has the following UTF schemes:

- **UTF-8 Encoding**

- The UTF8 is defined by the UNICODE standard, which is variable-width character encoding used in Electronics Communication.
- UTF-8 is capable of encoding all 1,112,064 valid character code points in Unicode using one to four one-byte (8-bit) code units.

- **UTF-16 Encoding**

UTF16 Encoding represents a character's code points using one of two 16-bits integers.

- **UTF-32 Encoding**

UTF32 Encoding represents each code point as 32-bit integers.

- Base64 Encoding

- Base64 Encoding is used to encode binary data into equivalent ASCII Characters.
- The Base64 encoding is used in the Mail system, as mail systems such as SMTP can't work with binary data because they accept ASCII textual data only.
- It is also used in simple HTTP authentication to encode the credentials.
- Moreover, it is also used to transfer the binary data into cookies and other parameters to make data unreadable to prevent tampering.
- If an image or another file is transferred without Base64 encoding, it will get corrupted as the mail system is not able to deal with binary data.
- Base64 represents the data into blocks of 3 bytes, where each byte contains 8 bits; hence it represents 24 bits.
- These 24 bits are divided into four groups of 6 bits. Each of these groups or chunks are converted into equivalent Base64 value.

- ASCII Encoding

- **American Standard Code for Information Interchange** (ASCII) is a type of character-encoding. It was the first character encoding standard released in the year 1963.
- The ASCII code is used to represent English characters as numbers, where each letter is assigned with a number from **0 to 127**.
- Most modern character-encoding schemes are based on ASCII, though they support many additional characters. It is a single byte encoding only using the bottom 7 bits.
- In an ASCII file, each alphabetic, numeric, or special character is represented with a 7-bit binary number.
- Each character of the keyboard has an equivalent ASCII value.